

day 16 | 251003 F

hitherto

We left off last time with a little bit of a question mark! Does decreasing the step size always increase the accuracy? Does it?

herein

We will look at the question of whether decreasing the step size of a derivative always increases the accuracy (spoiler: it doesn't, and its complicated).

We will also look into methods for differentiating DATA, which is similar and easy, but there are some "gotchas" there as well that we need to be aware of.

hence

We will move on to root finding methods, that is where does a function or data set cross a boundary of zero?

```
In [18]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
In [21]: def back(function, x_value, stepSize):
    return (function(x_value) - function(x_value - stepSize)) / stepSize

def forward(function, x_value, stepSize):
    return (function(x_value + stepSize) - function(x_value)) / stepSize

def mid(function, x_value, stepSize):
    return (function(x_value + stepSize) - function(x_value - stepSize)) / (2 * stepSize)

def doubled(function, x_value, stepSize):
    return (function(x_value + stepSize) - 2 * function(x_value) + function(x_value -
```

```
In [24]: def f(x):
    return np.exp(np.sin(x**2))

def g(x):
    return (2 * x * np.exp(np.sin(x**2)) * np.cos(x**2))

stepsizes = []

for i in range(20):
    stepsizes.append(10**(-i))
```

```

errorlist = []

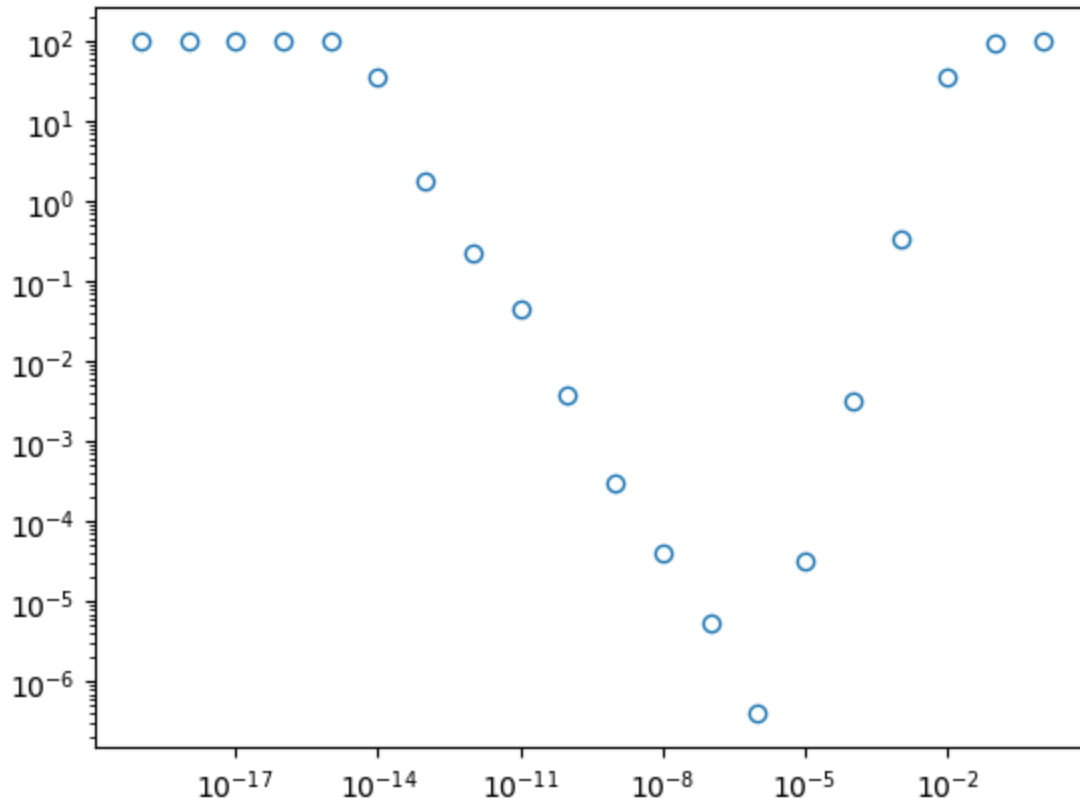
for i in stepsizes:
    d = mid(f, 67, i)
    e = abs((d-g(67))/g(67))*100
    errorlist.append(e)

fig, ax = plt.subplots()

ax.loglog(stepsizes, errorlist, 'o', mfc='none')

```

Out[24]: [



```

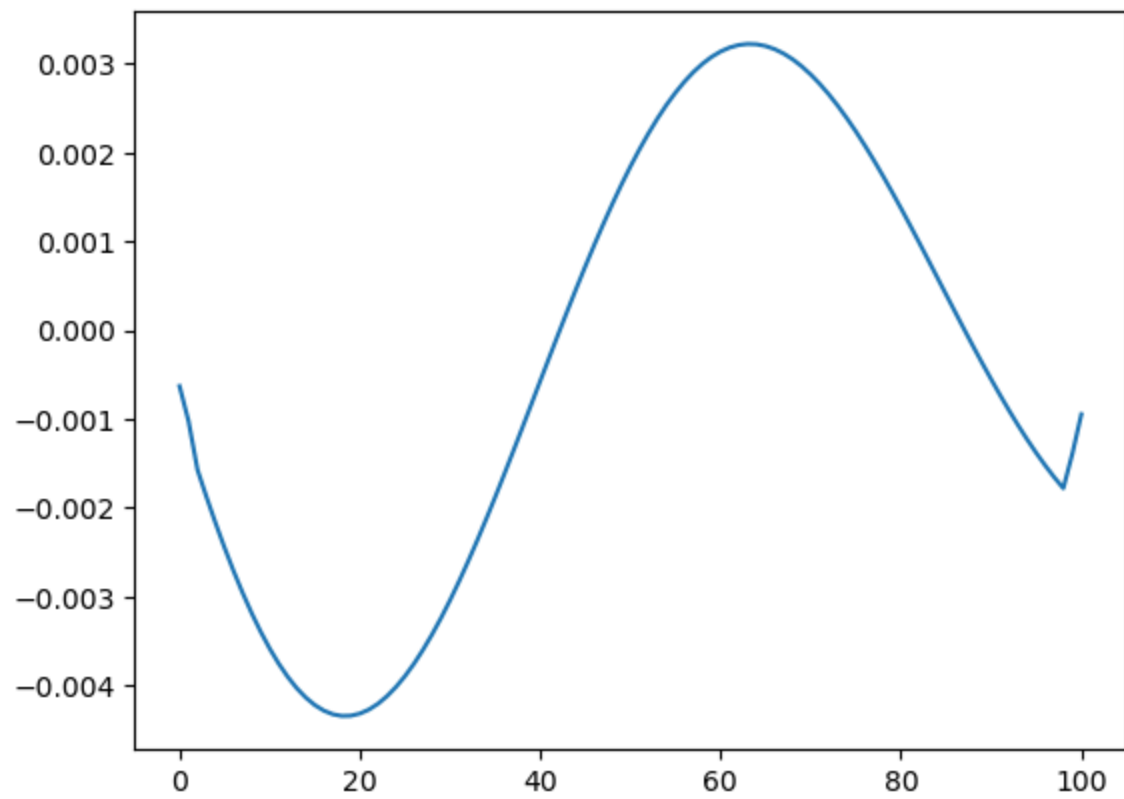
In [28]: data = np.loadtxt('../data/velocities.txt')
         t, v = data.T

         j = np.gradient(np.gradient(v, t), t)

         fig, ax = plt.subplots()
         #ax.plot(t, v)
         #ax.plot(t, a)
         ax.plot(t, j)

```

Out[28]: [



In []: